To follow along with the exercises in this section, you must register for a free account from TMDB and create an API key. Instructions for getting an API key can be found at the TMDB website (www.themoviedb.org) or by asking a GPT-4 turbo or a more recent LLM.

Over the next set of subsections, we'll create a GPT interface using an SK set of native functions. Then, we'll use the SK kernel to test the interface and, later in this chapter, implement it as plugins into a chat function. In the next section, we look at building a GPT interface against the TMDB API.

5.5.1 Building a semantic GPT interface

TMDB is an excellent service, but it provides no semantic services or services that can be plugged into ChatGPT or an agent. To do that, we must wrap the API calls that TMDB exposes in a semantic service layer.

A semantic service layer is a GPT interface that exposes functions through natural language. As discussed, to expose functions to ChatGPT or other interfaces such as agents, they must be defined as plugins. Fortunately, SK can create the plugins for us automatically, given that we write our semantic service layer correctly.

A native plugin or set of skills can act as a semantic layer. To create a native plugin, create a new plugin folder, and put a Python file holding a class containing the set of native functions inside that folder. The SK extension currently doesn't do this well, so manually creating the module works best.

Figure 5.9 shows the structure of the new plugin called `Movies` and the semantic service layer called `tmdb.py`. For native functions, the parent folder's name (`Movies`) is used in the import.